Цель

Создать Content-Based систему, которая находит музыку, похожую на любимый трек пользователя.

Строим прототип Content-Based рекомендательной системы

Описываем каждый трек вектором аудиопризнаков

Ищем ближайших соседей (Nearest Neighbors)

Таргетная колонка не нужна:
мы не обучаем модель предсказывать ответ



Датасет

Spotify Tracks Dataset

114 000
строк

114
жанров

Данные получены из **Spotify API**

В таблице есть **track_name, artists, album_name, explicit**

Аудиопризнаки описівают звучание композиции

Поле **popularity** оценивает популярность от 0 до 100



Источник: https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset

Как звук превращается в признаки

Spotify приобрёл The Echo Nest в 2014 году

Типовой подход: мел-спектрограмма → CNN → рекуррентные блоки

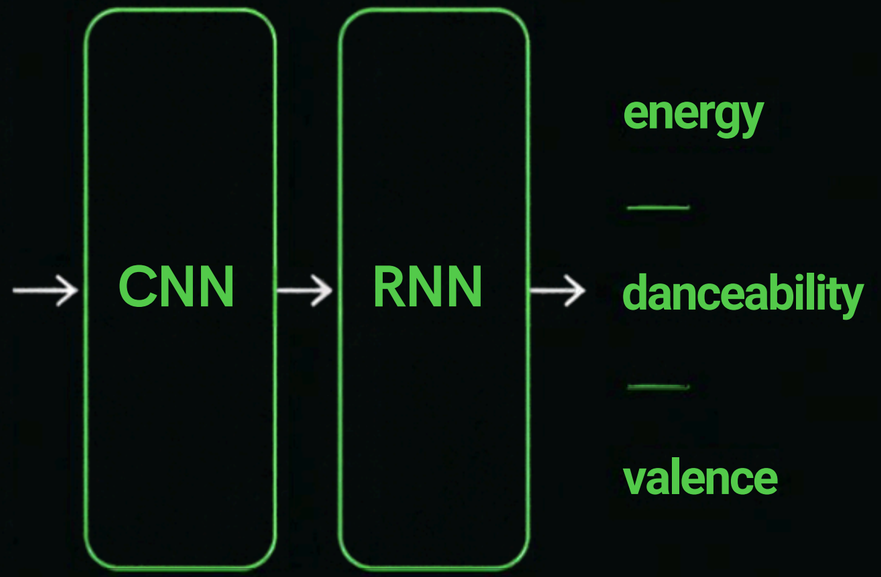
Модели анализируют спектр, ритм и гармонию

Energy, danceability и valence описывают восприятие звучания

Точные формулы Spotify не раскрыты



Спектрограмма



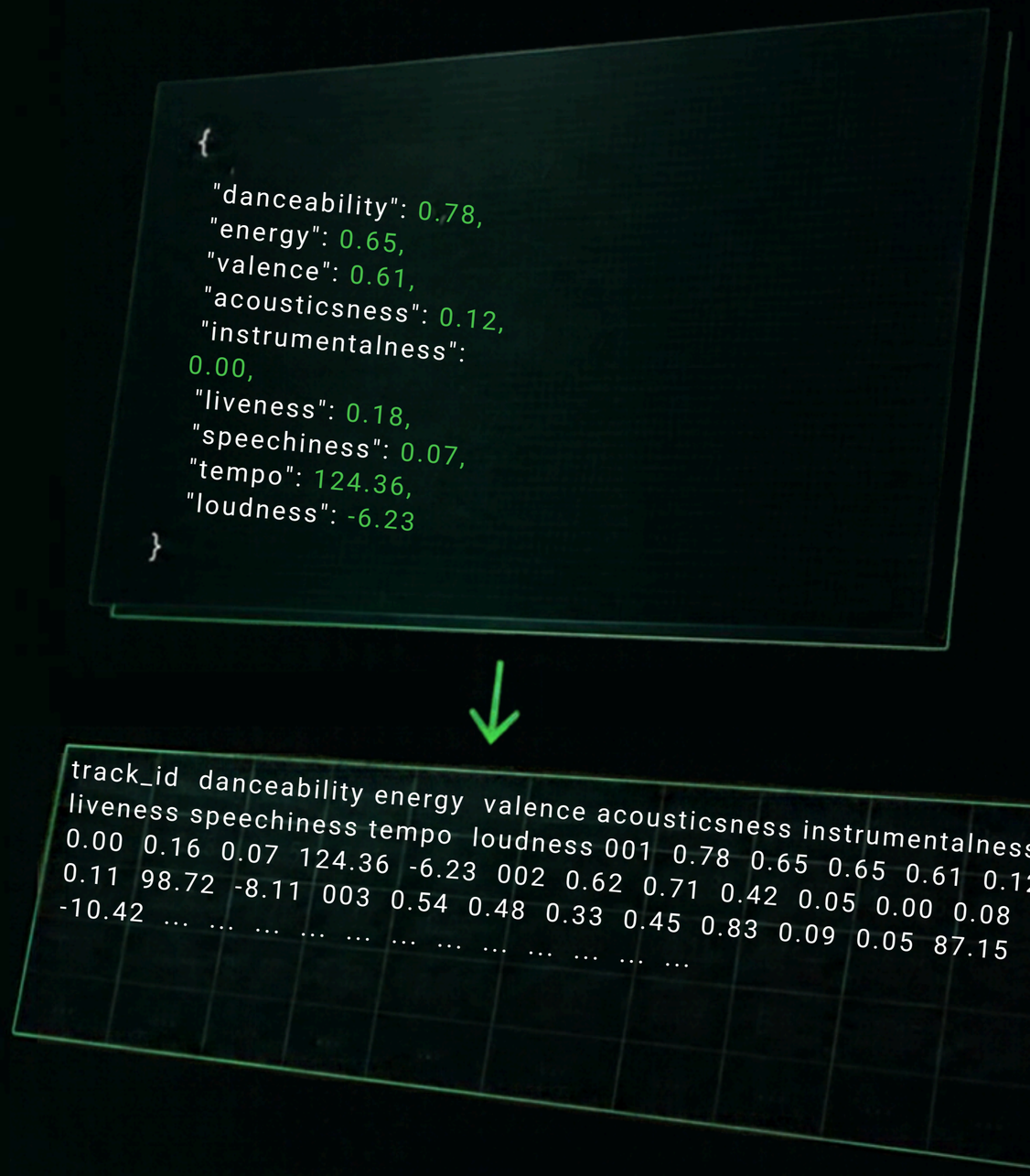
Формат входных Audio Features

В API Spotify Audio Features представлены как **JSON** с числовыми полями

Семь показателей имеют **диапазон 0-1:** danceability, energy, valence, acousticsness, instrumentalness, liveness, speechiness

Tempo измеряется в ударах в минуту **(BPM)**, loudness — в децибелах **(дБ)**

Используем готовый **CSV** и приводим признаки к сопоставимым масштабам



EDA

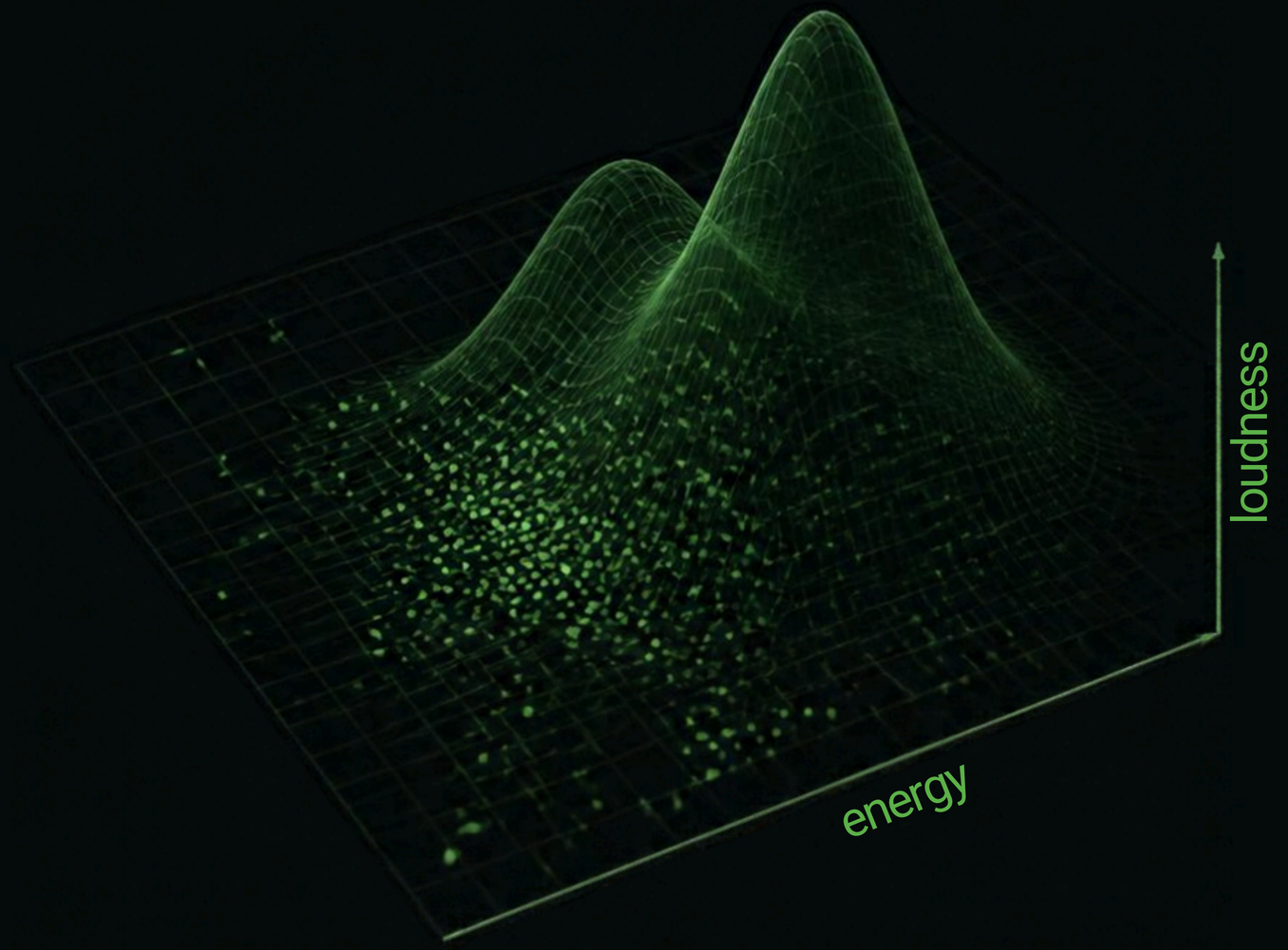
качество данных и связи признаков

Дубликаты track_id и пропуски

Распределения, выбросы и диапазоны

Корреляция energy и loudness: учитываем вес признаков, избегаем дублей

Popularity — для ранжирования и фильтра известности, а не свежести



Предобработка

Объединяем жанры одного track_id и
удаляем повторные строки

Обрабатываем **пропуски**
и проверяем корректность значений

MinMaxScaler приводит tempo
и loudness к диапазону 0–1

Альтернатива: **StandardScaler**
для всего числового блока

Опционально **кодируем жанры**:
One-Hot для одной метки,
Multi-Hot для нескольких



Архитектура

от запроса к топ-5

01

ввод

ID любимого трека или выбранные характеристики

02

вектор

извлекаем вектор и применяем сохранённые преобразования

03

сходство

считаем Cosine Similarity с матрицей треков

04

фильтры

исключаем исходный трек, применяем фильтры

05

результат

возвращаем пять наиболее похожих треков

ТОП 5

похожие треки

Работающий прототип без истории прослушиваний и без обучения собственной аудионейросети

При необходимости добавляем фильтр explicit и небольшую поправку на популярность